Τμήμα Μηχανικών Η/Υ & Πληροφορικής · Πανεπιστήμιο Πατρών

Αντικειμενοστρεφής Προγραμματισμός

Οδηγός μελέτης από τις παλιές εξεταστικές

CEID_23Y106 · 2ο εξάμηνο · υποχρεωτικό · 9 ECTS

Επιμέλεια: Giorgos Kandilas

https://kdngiorgos.github.io/ceid-notes/

Βάση γνώσης CEID · έκδοση 2026.10 · έκδοση 1.0

DOI: 10.5281/zenodo.23434967

CC BY-NC-SA 4.0 για το κείμενο του οδηγού· τα θέματα ανήκουν στους διδάσκοντες του μαθήματος.

Περιεχόμενα

Πριν ξεκινήσεις 2

1 Το μάθημα σε μία σελίδα 2

2 Χάρτης των εξετάσεων 3

3 Κλάσεις, αντικείμενα και κατασκευαστές 5

4 Αναφορές, ισότητα και αλφαριθμητικά 10

5 Κληρονομικότητα 14

6 Συλλογές Java 17

7 Αριθμητικοί τύποι και πίνακες 20

8 Πολυμορφισμός 23

9 Θεωρία και ερωτήσεις πολλαπλής επιλογής 27

10 Μνήμη, δημιουργία και καταστροφή αντικειμένων 29

11 Αναφορές και συναρτήσεις (C++) 32

12 Τελεστές και friend συναρτήσεις 34

13 Εξαιρέσεις 36

14 Πρότυπα (templates) και γενικεύσεις 38

15 Ασκήσεις εξάσκησης 40

Πηγές και ευχαριστίες 43

Πριν ξεκινήσεις

Αυτός ο οδηγός είναι φτιαγμένος από τις **8 εξετάσεις** εξετάσεων της βάσης γνώσης CEID (2020 έως 2026), το οδηγό του μαθήματος, τις φωτογραφίες και τα PDF του φακέλου και ό,τι έγραψαν φοιτητές στο Discord. Καμία εξέταση δεν είναι πλήρως μεταγεγραμμένη: όλες είναι **μερικές μεταγραφές** ή φωτογραφίες, και σε μερικές λείπουν σελίδες. Γι' αυτό τα ποσοστά του χάρτη είναι **κάτω όρια**, όχι ακριβείς αναλογίες.

Σημείωση

Διαφορά από άλλα μαθήματα: οι λύσεις είναι λίγες και **δεν είναι επίσημες**. Υπάρχουν **κοινοτικές** λύσεις για τις εξετάσεις Ιουνίου 2021 και Ιουνίου 2022 (✓), και **λύσεις AI** για τις εξετάσεις 2020, 2023 (και τα ενδεικτικά), 2025 (Ιούνιος και Σεπτέμβριος) και 2026 (⚙). Όσες λύσεις AI υπάρχουν εδώ δεν έχουν ελεγχθεί από άνθρωπο και σημειώνονται AI. Λύσ' τα πρώτα μόνος σου.

Τι σημαίνουν οι σημάνσεις

- ✓ — κοινοτική λύση (φοιτητές), από το αντίστοιχο αρχείο `.solutions.md` της βάσης. Δεν είναι επίσημη.
- ⚙ — λύση AI, χωρίς επίσημη λύση για σύγκριση. AI
- ○ — χωρίς λύση. Υπάρχει μόνο η μέθοδος, χωρίς τελική απάντηση.
- DISCORD — γνώμη φοιτητών από το Discord του CEID, με ημερομηνία. Χρήσιμη, αλλά ανεπίσημη.

1 Το μάθημα σε μία σελίδα

Τι είναι. Εισαγωγή στο αντικειμενοστρεφές παράδειγμα προγραμματισμού με Java (και μετάβαση σε C++): κλάσεις και αντικείμενα, κατασκευαστές, κληρονομικότητα, πολυμορφισμός, εξαιρέσεις, συλλογές, UML, ενώ μέρος της ύλης αφορά γραφικές διεπαφές, νήματα και δικτύωση. Οι διαλέξεις του eClass δίνουν τη σειρά· η C++ εμφανίζεται προς το τέλος ως μετάβαση για όσους ξέρουν ήδη Java.

Διδάσκοντες 2025-26. Π. Χατζηδούκας, Α. Κομνηνός και Ι. Περίκος (Εντεταλμένος Διδάσκων) (κατάλογος μαθημάτων 2025-26).

Ώρες και πίστωση. 3 ώρες θεωρία, 2 ώρες φροντιστήριο, 2 ώρες εργαστήριο· 9 ECTS (κατάλογος 2025-26).

Πώς βαθμολογείται. Το επίσημο περίγραμμα αναφέρει τελική εξέταση 50–70% και εργαστηριακές ασκήσεις 30–50%. Ο τρόπος ανακοινώνεται στην αρχή του εξαμήνου. Κατά φοιτητές, για 2025-26 ακούγονται δύο εκδοχές, 100% τελική ή 70% τελική και 30% εργαστήριο DISCORD 2026-06. Για το εργαστήριο στο νέο καθεστώς δεν βρέθηκαν πληροφορίες για το ακριβές ποσοστό.

Η εξέταση.

- **Μορφή.** Στις παλιές εξετάσεις (2020–2023) η εξέταση ήταν πολλαπλής επιλογής στο υπολογιστικό κέντρο, ή, το 2020, ανοιχτή ανάπτυξη κώδικα («Click Away»). Από το 2025 είναι γραπτή, με πολλαπλής επιλογής με αιτιολόγηση, μικρά τμήματα κώδικα («τι εκτυπώνει ο κώδικας») και ένα-δύο θέματα υλοποίησης, κυρίως σε Java, με ένα θέμα C++ (στις εξετάσεις 2025 και 2026).
- **Αυτό που λένε φοιτητές για τη γλώσσα.** «Είπε και ο Χατζηδούκας τα περισσότερα πολλαπλής θα είναι στην Java» DISCORD 2025-06-13.
- **Τι ζητάει η εξέταση, κατά φοιτητές.** «η εξέταση έχει πολλαπλής» και «και να συμπληρώσεις τι εκτυπώνει ο κώδικας» DISCORD 2026-06-03.

- **Τι ξεχώρισε στο εργαστήριο, κατά φοιτητή.** «Στο εργαστήριο όμως έπαιξε πολύ με: Abstract Classes, Inheritance, Exceptions, File Handling, Methods(private, public, protected)» DISCORD 2025-06-13. Εκτός από τα αρχεία (file handling), τα υπόλοιπα αντιστοιχούν σε ενότητες του χάρτη παρακάτω.
- **Διάρκεια.** 2 ώρες και 30 λεπτά στο θέμα του Ιουνίου 2025 (τυπωμένο στην κεφαλίδα). Για τα υπόλοιπα δεν βρέθηκαν πληροφορίες, εκτός από το Ενδεικτικό τεστ (10 λεπτά) και την εξέταση 2020 (65 λεπτά).
- **Τυπολόγιο.** Δεν βρέθηκαν πληροφορίες.
- **Υλικό.** Επιτρέπεται σημειώσεις ή βιβλίο; Σύμφωνα με τις ανακλήσεις φοιτητών για την εξέταση του 2025, όχι. Δεν βρέθηκε πρωτότυπη ανακοίνωση.

Σημείωση

Τι να μελετήσεις πρώτα. Ξεκίνα από τις **κλάσεις και τους κατασκευαστές** (κεφ. 1), τα **αλφαριθμητικά και την ισότητα** (κεφ. 2) και την **κληρονομικότητα** (κεφ. 3). Αυτά τα τρία καλύπτουν περίπου το ένα τρίτο των ερωτήσεων. Μετά πέρνα τις **συλλογές της Java** (κεφ. 4), τους **τύπους και τους πίνακες** (κεφ. 5) και τον **πολυμορφισμό** (κεφ. 6). Οι εξαιρέσεις και τα templates είναι μικρότερες ενότητες, αλλά εμφανίστηκαν στις τρεις τελευταίες γραπτές εξετάσεις.

2 Χάρτης των εξετάσεων

Ο πίνακας μετρά **μοναδικές ερωτήσεις** στις 8 εξετάσεις (2020, Ιούνιος 2021, Ιούνιος 2022, Ιούνιος 2023, το ενδεικτικό τεστ, Ιούνιος 2025, Σεπτέμβριος 2025, Ιούνιος 2026). Η μέτρηση έγινε με το χέρι από τις μεταγραφές. Κάθε υποερώτημα (a–d ή i–v) μετράει ως ξεχωριστή ερώτηση. Δεν περιλαμβάνονται οι ερωτήσεις που λείπουν από τις φωτογραφίες (π.χ. η 3η του ενδεικτικού τεστ προγραμματισμού και η σελίδα 3/4 του Ιουνίου 2025).

Θεματική ενότητα	Ερωτήσεις	Εξετάσεις (από 8)	Κεφ.
Κλάσεις, αντικείμενα και κατασκευαστές	**15**	6	1
Αναφορές, ισότητα και αλφαριθμητικά	**11**	5	2
Κληρονομικότητα	**11**	5	3
Συλλογές Java	**10**	3	4
Αριθμητικοί τύποι και πίνακες	**9**	6	5
Πολυμορφισμός	**9**	4	6
Θεωρία και ερωτήσεις πολλαπλής επιλογής	**8**	4	7
Μνήμη, δημιουργία και καταστροφή αντικειμένων	**7**	6	8
Αναφορές και συναρτήσεις (C++)	**6**	4	9
Τελεστές και friend συναρτήσεις	**4**	3	10
Εξαιρέσεις	**3**	3	11
Πρότυπα (templates) και γενικεύσεις	**2**	2	12

Σημείωση

Σημείωση για τον χάρτη. Η εξέταση 2020 έχει μία ερώτηση (σχεδίαση της εφαρμογής «Click Away»). Το ενδεικτικό τεστ του 2023 δεν είναι βαθμολογούμενη εξέταση, αλλά μετράει στον χάρτη. Το θέμα «Ιούνιος 2025» έχει την 3η σελίδα λείπουσα.

Χρονολόγιο: πόσες ερωτήσεις ανά ενότητα και έτος (κενό = δεν εμφανίστηκε)

Ενότητα	’20	’21	’22	’23	’25	’26	Σ
Κλάσεις, αντικείμενα και κατασκευαστές	3	5		6	1	4	**19**
Θεωρία και ερωτήσεις πολλαπλής επιλογής		1	2	5	1		**9**
Πολυμορφισμός		3		4			**7**
Μνήμη, δημιουργία και καταστροφή αντικειμένων				5			**5**
Κληρονομικότητα		1		3			**4**
Αναφορές και συναρτήσεις (C++)		2		1			**3**
Αναφορές, ισότητα και αλφαριθμητικά		1		1			**2**
Αριθμητικοί τύποι και πίνακες		1		1			**2**
Τελεστές και friend συναρτήσεις		1					**1**
Πρότυπα (templates) και γενικεύσεις		1					**1**

Τύποι ερωτήσεων (53 ερωτήσεις, 50 από το 2021 και μετά)

Τύπος	Όλα	%	Από 2021
Πολλαπλή επιλογή	49	92%	49
Σχεδίαση / εκτέλεση αλγορίθμου	3	6%	0
Υπολογισμός	1	2%	1

Τα δύο παραπάνω σχήματα παράγονται αυτόματα από την ταξινόμηση των ερωτήσεων· μία ερώτηση με υποερωτήματα μετράει μία φορά.

3

Κλάσεις, αντικείμενα και κατασκευαστές

Μέθοδος

(μέθοδος με δικά μας λόγια· βλ. eClass, διαλέξεις για κλάσεις και αντικείμενα)
Κατασκευαστές. Ο κατασκευαστής τρέχει όταν δημιουργείται το αντικείμενο. Για να μην επαναλαμβάνεις κώδικα, γράψε τον **πλήρη** κατασκευαστή μία φορά και τους άλλους τον καλούν με `this(...)` (Java) ή με αρχικοποίηση στη λίστα (C++). Η `this(...)` πρέπει να είναι η **πρώτη** εντολή του κατασκευαστή.
Αρχικοποίηση μελών. Σε C++, τα `const` μέλη και τα μέλη τύπου αναφοράς αρχικοποιούνται **μόνο** στη λίστα αρχικοποίησης (`: id(i)`). Η ανάθεση μέσα στο σώμα δεν αρκεί. Ένα `const` πεδίο χωρίς αρχικοποίηση στη λίστα δεν μεταγλωττίζεται.
Στατικά μέλη. Ένα `static` πεδίο είναι ένα για όλη την κλάση, όχι ένα ανά αντικείμενο. Μετράς τις δημιουργίες με `static` μετρητή που αυξάνεται στον κατασκευαστή.
this. Το `this` ξεχωρίζει το πεδίο από την τοπική μεταβλητή με το ίδιο όνομα. Γράψε `this.code` για το πεδίο, όχι για την τοπική.
Ιδιωτικά μέλη. Το `private` πεδίο είναι προσβάσιμο μόνο μέσα στην κλάση. Η `main` σε άλλη κλάση που γράφει `c.count` δίνει σφάλμα μεταγλώττισης.

Παράδειγμα: κατασκευαστές με `this(...)` (Ιούνιος 2026, Θέμα 3)

Θέμα εξέτασης

Θέμα 3ο (2 μονάδες): Υλοποιήστε την κλάση Course ώστε να υποστηρίζεται ο παρακάτω κώδικας. Αποφύγετε την επανάληψη κώδικα στους κατασκευαστές. Απαντήστε στο χώρο που δίνεται.

```
Course c1 = new Course("Java");
Course c2 = new Course("C++", "K. Papadopoulos");
Course c3 = new Course("Data Structures", "M. Georgiou", 6);

c1.enroll();
c1.enroll();

c1.display();
// Title: Java, Teacher: Unknown, ECTS: -1, Students: 2

c2.display();
// Title: C++, Teacher: K. Papadopoulos, ECTS: -1, Students: 0

c3.display();
// Title: Data Structures, Teacher: M. Georgiou, ECTS: 6, Students: 0
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Η μέθοδος είναι να έχει ο πλήρης κατασκευαστής τρεις παραμέτρους και οι άλλοι να τον καλούν με `this(...)`, με τις προεπιλογές `"Unknown"` και `-1`:

```
public class Course {
```

```
    private String title;
    private String teacher;
    private int ects;
    private int students;

    public Course(String title) { this(title, "Unknown"); }
   public Course(String title, String teacher) { this(title, teacher, -1); }
    public Course(String title, String teacher, int ects) {
        this.title = title;
        this.teacher = teacher;
        this.ects = ects;
        this.students = 0;
    }
    public void enroll() { students++; }
    public void display() {
        System.out.println("Title: " + title + ", Teacher: " + teacher
                + ", ECTS: " + ects + ", Students: " + students);
    }
}
```

Η λύση δίνει τις τρεις γραμμές των σχολίων, όπως ζητά η εκφώνηση. Η απάντηση ελέγχθηκε μόνο με το χέρι.

Παράδειγμα: στατικός μετρητής και κατασκευαστής (Ιούνιος 2023, Java 4/10)

Θέμα εξέτασης

Δίνεται το παρακάτω πρόγραμμα. Τί θα εκτυπωθεί μετά την εκτέλεσή της main;

```
class Window
{ public int size = 1;
  public static int count = 0;
  public Window()
  {this.size = 3;
   setCount ();}
  public static void setCount()
   {count++;}
class XWindow extends Window
  public XWindow(int x)
   {this.size = --x;}
public class RunWindow
{ public static void main ()
  {
   XWindow w1 = new XWindow(6);
   Window w = new Window();
   System.out.println(w1.size);
   System.out.println(w.count);
  }}
```

□ 6 0

- ☐ 5 0
- ☐ 5 2
- ☐ 6 2

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **5 και 2** (η τρίτη επιλογή).
Ο κατασκευαστής του `XWindow` καλεί πρώτα (σιωπηρά) τον `Window()`, άρα το `count` γίνεται 1. Το `--x` δίνει 5, που αποθηκεύεται στο `size`. Το `new Window()` αυξάνει το `count` σε 2, και επειδή το `count` είναι στατικό, το `w.count` δείχνει 2. Η εκφώνηση έχει λείπουσα παρένθεση στο `class Window`, που θεωρήθηκε τυπογραφικό.

Σημείωση

Συχνό λάθος: να ξεχνάς ότι ο κατασκευαστής της υπερκλάσης τρέχει πρώτος, ακόμη και όταν δεν τον γράφεις. Γι' αυτό ένα `static` μετρητής που αυξάνεται εκεί μετράει και τα αντικείμενα της υποκλάσης.

Παράδειγμα: πεδίο και `this` (Ιούνιος 2021, Ερώτηση 12 / 40)

Θέμα εξέτασης

Δίνεται ο παρακάτω κώδικας . Με την εκτέλεσή του θα εμφανιστεί στην οθόνη :

```
class Exams2
{
    int code=40;
    void printCode()
    {
        int code =50;
        System.out.println("Code: "+this.code);
    }
    public static void main (String[] args){
        Exams2 e2=new Exams2();
        e2.printCode();}
}
```

- ☐ Code: 40
- ☐ Code: 50
- ☐ Code: 90

Λύση

✓ **Κοινοτική λύση.** Απάντηση: **Code: 40**. Το `this.code` αναφέρεται στο πεδίο, ενώ η τοπική `code` σκιάζει μόνο το όνομα μέσα στη μέθοδο (`2021-06.solutions.md`).

Ακόμη λυμένα θέματα

3.4.1 Σεπτέμβριος 2025, Θέμα 2β: `private` πεδίο σε άλλη κλάση

Θέμα εξέτασης

b.

```
class Counter {
    private int count;
    public void increment() {
        count++;
    }
}
public class Main
{
    public static void main(String[] args) {
        Counter c = new Counter();
        c.increment();
        System.out.println(c.count);
    }
}
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Η λύση αφορά ελαφρώς διαφορετικό κώδικα (χωρίς `increment`) και συμπεραίνει σφάλμα μεταγλώττισης, επειδή το `private` πεδίο είναι προσβάσιμο μόνο μέσα στην `Counter`. Το συμπέρασμα ισχύει και για τον κώδικα του θέματος, όπως σημειώνει το αρχείο λύσεων.

3.4.2 Ιούνιος 2026, Θέμα 5γ: `const` μέλος και λίστα αρχικοποίησης

Θέμα εξέτασης

c. Έστω κλάση Student με πεδία:

```
string name;
int semester;
const int id;
```

Ποιος είναι ο σωστός κατασκευαστής (constructor) για την αρχικοποίηση ενός αντικειμένου; 1. Student::Student(string n, int s, int i) : name(n), semester(s), id(i) { } 2. Student::Student(string n, int s, int i) { name = n; semester = s; id = i; } 3. Student::Student(string n, int s) : name(n), semester(s) { id = 0; } 4. Ο 1ος και ο 2ος

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **η 1η**. Το `const int id` πρέπει να αρχικοποιηθεί στη λίστα αρχικοποίησης μελών και δεν μπορεί να πάρει τιμή στο σώμα. Η 2η δεν

μεταγλωττίζεται για τον ίδιο λόγο.

4

Αναφορές, ισότητα και αλφαριθμητικά

Μέθοδος

(μέθοδος με δικά μας λόγια· βλ. eClass, διαλέξεις για String και αναφορές)
Αναφορά ή τιμή. Στη Java, μια μεταβλητή τύπου κλάσης είναι **αναφορά**. Η `w1 = w2` δεν αντιγράφει το αντικείμενο, αλλά κάνει τη `w1` να δείχνει σε ό,τι δείχνει η `w2`. Οι αναφορές δείχνουν στο ίδιο αντικείμενο, και η αλλαγή μέσω της μίας φαίνεται και στην άλλη.
`==` και `equals`. Το `==` συγκρίνει **διευθύνσεις** (αν δύο αναφορές δείχνουν στο ίδιο αντικείμενο). Το `equals` συγκρίνει **περιεχόμενο**. Δύο `new` αντικείμενα ίδιου περιεχομένου δίνουν `==` false και `equals` true (αν η κλάση ορίζει `equals`).
Αλφαριθμητικά. Το `String` είναι **αμετάβλητο**. Η `replace` επιστρέφει νέο `String` και δεν αλλάζει το υπάρχον. Μια σταθερά όπως `"Ja" + "va"` ενώνεται κατά τη μεταγλώττιση και μοιράζεται το ίδιο αντικείμενο με το literal `"Java"`, ενώ το `new String("Java")` είναι πάντα νέο αντικείμενο.
Μήκος. Το `s.length()` είναι μέθοδος του `String`. Το `arr.length` είναι πεδίο του πίνακα. Μην τα μπερδεύεις.

Παράδειγμα: `==` σε literal και `new String` (Ιούνιος 2026, Θέμα 1γ)

Θέμα εξέτασης

Θέμα 1ο (2 μονάδες): Ποια είναι η έξοδος στις παρακάτω περιπτώσεις; Απαντήστε και δώστε σύντομη εξήγηση. Αν ο κώδικας δεν μεταγλωττίζεται, εξηγήστε γιατί.
c.

```
String s1 = "Java";
String s2 = "Ja" + "va";
String s3 = new String("Java");
System.out.println(s1 == s2);
System.out.println(s1 == s3);
System.out.println(s2.equals(s3));
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **true, false, true**. Το `"Ja" + "va"` είναι σταθερά μεταγλώττισης και δείχνει στο ίδιο literal· το `new String` δημιουργεί νέο αντικείμενο· και τα δύο έχουν το ίδιο περιεχόμενο.

Παράδειγμα: ισότητα και `equals` στη σύγκριση αλφαριθμητικών (Ιούνιος 2021, Ερώτηση 11 / 40)

Θέμα εξέτασης

Η παρακάτω μέθοδος θέλουμε να συγκρίνει δύο αλφαριθμητικά και να δίνει κατάλληλη έξοδο. Πρέπει να συμπληρωθεί ο κώδικας στα σημεία (1) και (2). Ποια από τις παρακάτω δυάδες είναι η σωστή;

```
public (1) compString(String s1, String s2)
{
```

```
    boolean result = (2);
    {if result
        System.out.println ("Τα αλφαριθμητικά είναι ίσα.");}
     else System.out.println ("Τα αλφαριθμητικά δεν είναι ίσα.");
}
```

- ☐ (1): boolean, (2): s1.equals(s2)
- ☐ (1): void, (2): s2.equals(s1)
- ☐ (1): boolean, (2): equals(s1,s2)
- ☐ (1): void, (2): equals(s1,s2)

Λύση

✓ **Κοινοτική λύση.** Απάντηση: **(1): void, (2): s2.equals(s1)**. Η μέθοδος τυπώνει και δεν επιστρέφει τιμή, άρα το `void`. Το `equals` καλείται πάνω στο ένα αλφαριθμητικό και παίρνει το άλλο ως όρισμα (`2021-06.solutions.md`). Η εκφώνηση έχει λείπουσα παρένθεση στο `if`, που το αρχείο λύσεων δεν σχολιάζει.

Σημείωση

Συχνό λάθος: να κρίνεις το `==` σε αλφαριθμητικά από την ίδια τιμή. Το `"Hello" == new String("Hello")` είναι false, παρότι το `equals` είναι true.

Παράδειγμα: σύγκριση αλφαριθμητικών με `length` (Ιούνιος 2023, Java 9/10)

Θέμα εξέτασης

Η παρακάτω μέθοδος της κλάσης ManageStrings συγκρίνει δύο αλφαριθμητικά και επιστρέφει κατάλληλο αποτέλεσμα. Στη main εκτελείται η εντολή: String label = ms1.compString("Hello", "Ioannis"), όπου ms1 στιγμιότυπο της ManageStrings. Ποια θα είναι η τιμή της μεταβλητής label;

```
public String compString(String s1, String s2)
{
    boolean result = s1.equals(s2);
    if !result
        if (s1.length() > s2.length())
            return s1+s2;
        else return s2+s1;
    else return s1;
}
```

- ☐ "HelloIoannis"
- ☐ "IoannisHello"
- ☐ "Hello"
- ☐ IoannisHello

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **"IoannisHello"** (2η επιλογή). Ο κώδικας όπως μεταγράφηκε έχει `if  !result` χωρίς παρενθέσεις, που δεν μεταγλωττίζεται. Με τη διόρθωση `if (!result)` τα μήκη είναι 5 και 7, άρα επιστρέφεται `s2+s1`. Η εκφώνηση μάλλον έχει τυπογραφικό λάθος, γι' αυτό η απάντηση είναι υπό επιφύλαξη.

Ακόμη λυμένα θέματα

4.4.1 Ιούνιος 2026, Θέμα 2β: `Test.count` και static μεταβλητή

Θέμα εξέτασης

b.

```
class Test {
    static int count = 1;
    Test() { count *= 2; }
}

Test a = new Test();
Test b = new Test();
System.out.println(Test.count);
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **4**. Το `count` είναι στατικό, άρα πολλαπλασιάζεται από κάθε δημιουργία: $1 \to 2 \to 4$.

4.4.2 Ιούνιος 2023, Java 2/10: `charCount` με `charAt`

Θέμα εξέτασης

Η παρακάτω μέθοδος θέλουμε να μετρά και να επιστρέφει τον αριθμό των εμφανίσεων του χαρκτήρα c στο αλφαρηθμητικό s. Πρέπει να συμπληρωθεί ο κώδικας στα σημεία (1) και (2). Ποια από τις παρακάτω δυάδες είναι η σωστή. Πρέπει να συμπληρωθεί ο κώδικας στα σημεία (1) και (2). Ποια από τις παρακάτω δυάδες είναι η σωστή.

```
public int charCount (String s, char c)
{
    int count=0;
    for (int i=0; (1); i++)
        {if ((2))
            count++;}
    return count;
}
```

- ☐ (1): i <= s.length(), (2): c==s.charAt(i)
- ☐ (1): i <= s.length, (2): c==s.charAt(i)

- ☐ (1): i < s.length(), (2): c==s.charAt(i)
- ☐ i < s.length(), (2): c==s.indexOf(c)

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **η 3η επιλογή**, με $i < s.length()$ και `c == s.charAt(i)`. Το `length` είναι μέθοδος του `String`, και ο δείκτης πρέπει να φτάνει μέχρι length $- 1$.

5

Κληρονομικότητα

Μέθοδος

(μέθοδος με δικά μας λόγια· βλ. eClass, διαλέξεις για κληρονομικότητα)
Κατασκευαστές υποκλάσης. Ο κατασκευαστής της υποκλάσης καλεί **πρώτα** τον κατασκευαστή της υπερκλάσης. Αν δεν γράψεις `super(...)` ή `this(...)` στην αρχή, η Java καλεί σιωπηρά το `super()` χωρίς ορίσματα. Αν η υπερκλάση δεν έχει τέτοιον κατασκευαστή, έχεις σφάλμα μεταγλώττισης. Σε C++ αυτό γίνεται στη λίστα αρχικοποίησης.
Κρυμμένα πεδία. Στη Java, τα **πεδία** δεν πολυμορφίζονται. Αν η υποκλάση ορίζει πεδίο με το ίδιο όνομα, το πεδίο της υπερκλάσης κρύβεται. Ο **στατικός** τύπος της αναφοράς καθορίζει ποιο πεδίο παίρνεις.
Μέθοδοι και δείκτες. Η κλήση μεθόδου πρέπει να υπάρχει στον **στατικό** τύπο της αναφοράς. Η `Parent obj = new Child(); obj.onlyInChild();` δεν μεταγλωττίζεται αν η `onlyInChild` υπάρχει μόνο στην `Child`. Για να την καλέσεις χρειάζεται cast: `((Child) obj).onlyInChild()`.
Ανάθεση προς τα πάνω. Η `Student s = new MasterStudent();` είναι έγκυρη (upcast). Το αντίστροφο θέλει cast.

Παράδειγμα: κρυμμένο πεδίο (Ενδεικτικό 2023, πρόγραμμα 1ο)

Θέμα εξέτασης

Δίνεται το παρακάτω πρόγραμμα

```
class Window{
  double size;
  boolean dmode = false;
  public Window() {
    size = 2.0;}}
class EWindow extends Window{
  protected boolean dmode = true;
  public EWindow (double size){
    this.size = size--;}}
public class RunWindow{
  public static void main(){
    EWindow w1 = new EWindow(3.5);
    ...System.out.println(w1.dmode);
}}
```

Τί θα εκτυπωθεί μετά την εκτέλεσή του;

- ☐ true
- ☐ false
- ☐ null
- ☐ τίποτα

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **true** (1η επιλογή). Το πεδίο `dmode` της `EWindow` κρύβει το πεδίο της `Window`, και ο στατικός τύπος του `w1` είναι `EWindow`. Η εκφώνηση έχει `...` και `main()` χωρίς ορίσματα, που θεωρήθηκαν τυπογραφικά.

Σημείωση

Συχνό λάθος: να περιμένεις ότι τα πεδία αλλάζουν με τον τύπο του αντικειμένου, όπως οι μέθοδοι. Στη Java κοιτάς τον στατικό τύπο για πεδία.

Παράδειγμα: κλήση μεθόδου της υποκλάσης μέσω αναφοράς της υπερκλάσης (Ιούνιος 2026, Θέμα 2γ)

Θέμα εξέτασης

c.

```
class A {
    void f() { System.out.print("A"); }
}

class B extends A {
    void g() { System.out.print("B"); }
}

A obj = new B();
obj.g();
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Ο κώδικας **δεν μεταγλωττίζεται**: ο στατικός τύπος του `obj` είναι `A`, που δεν έχει μέθοδο `g()`. Η διόρθωση είναι `((B) obj).g()`.

Ακόμη λυμένα θέματα

5.3.1 Ιούνιος 2025, Θέμα 2δ: υποκλάση και καλούμενη μέθοδος

Θέμα εξέτασης

d.

```
class Parent {
    void print() { System.out.println("Parent"); }
}

class Child extends Parent {
    void print() { System.out.println("Child"); }
    void onlyInChild() { System.out.println("Child only"); }
```

```
}

public class Test {
    public static void main(String[] args) {
        Parent obj = new Child();
        obj.onlyInChild();
    }
}
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Δεν μεταγλωττίζεται: ο στατικός τύπος `Parent` δεν έχει την `onlyInChild`. Η λύση είναι cast σε `Child`, όπως στο παράδειγμα του κεφαλαίου.

5.3.2 Ιούνιος 2021, Ερώτηση 22 / 40: constructor της υπερκλάσης

Θέμα εξέτασης

Ο constructor της παραγόμενης κλάσης καλεί τον constructor της κλάσης βάσης:

- ☐ Άμεσα, μέσα στο σώμα του constructor
- ☐ Άμεσα, με super
- ☐ Άμεσα, στη member initialization list
- ☐ Μπορεί να καλέσει μόνο έμμεσα, τον default

Λύση

✓ **Κοινοτική λύση.** Απάντηση: **Άμεσα, με super** (`2021-06.solutions.md`). Στη Java η κλήση γίνεται με `super(...)`, ενώ στη C++ με τη λίστα αρχικοποίησης.

6

Συλλογές Java

Μέθοδος

(μέθοδος με δικά μας λόγια· βλ. eClass, διαλέξεις για συλλογές και δομές δεδομένων)

ArrayList. Το `add(θέση, στοιχείο)` μετατοπίζει τα επόμενα στοιχεία προς τα δεξιά. Το `remove(int)` αφαιρεί **με θέση**, ενώ το `remove(Object)` αφαιρεί **την πρώτη εμφάνιση** της τιμής και επιστρέφει `false` αν δεν υπάρχει (χωρίς εξαίρεση). Σε `ArrayList<Integer>` το `remove(i)` με `int i` αφαιρεί με θέση.

Βρόχος με αφαίρεση. Αν αφαιρείς μέσα σε βρόχο με δείκτη `i`, τα στοιχεία μετατοπίζονται και **παραλείπεις** κάποιο. Ο έλεγχος `i < list.size()` γίνεται ξανά κάθε φορά, άρα ο βρόχος δεν χτυπά εξαίρεση, απλώς δεν εξετάζει όλα τα στοιχεία.

LinkedList ως ουρά. Το `add` βάζει στο τέλος και το `remove()` αφαιρεί από την αρχή. Άρα η σειρά εξόδου είναι FIFO.

Stack. Το `push` βάζει στην κορυφή, το `pop` βγάζει από την κορυφή (LIFO). Το `pop` σε άδεια στοίβα ρίχνει `EmptyStackException`. Πρόσεξε τη σειρά των `push` σε κάθε βήμα του βρόχου.

HashSet. Δεν δέχεται διπλότυπα. Η επανεισαγωγή ίδιας τιμής δεν αλλάζει το `size()`.

Παράδειγμα: αφαίρεση μέσα σε βρόχο (Ιούνιος 2025, Θέμα 2β)

Θέμα εξέτασης

b.

```
ArrayList<Integer> list =
    new ArrayList<>(Arrays.asList(1, 8, 4, 11, 5));
for (int i = 0; i < list.size(); i++) {
    if (list.get(i) % 2 == 1) list.remove(i);
}
System.out.println(list);
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **[8, 4, 5]**. Το `1` αφαιρείται στο `i=0`, και το `11` αφαιρείται στο `i=2` (μετά τη μετατόπιση). Το `5` παραλείπεται στο `i=3`, επειδή το `size()` έχει γίνει 3.

Σημείωση

Συχνό λάθος: να υποθέτεις ότι ο βρόχος αφαιρεί όλα τα περιττά. Γράψε κάθε βήμα με τον πίνακα ως έχει μετά από κάθε αφαίρεση.

Παράδειγμα: ουρά με LinkedList (Σεπτέμβριος 2025, Θέμα 1α)

Θέμα εξέτασης

Θέμα 1ο (2 μονάδες): Ποια είναι η έξοδος στις παρακάτω περιπτώσεις; Απαντήστε και δώστε σύντομη εξήγηση.

a.

```
LinkedList<String> queue = new LinkedList<>();
queue.add("C");
queue.add("B");
queue.add("A");
System.out.println(queue.remove());
System.out.println(queue.remove());
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **C και B** σε δύο γραμμές. Το `remove()` αφαιρεί από την αρχή της ουράς, άρα πρώτα το `"C"`, μετά το `"B"`. Η λύση του αρχείου (ai-unverified) συμφωνεί με τη μέθοδο.

Ακόμη λυμένα θέματα

6.3.1 Ιούνιος 2026, Θέμα 1β: HashSet και μέγεθος

Θέμα εξέτασης

b.

```
HashSet<Integer> set = new HashSet<>();
set.add(2); set.add(4);
set.add(2); set.add(6);
System.out.println(set.size());
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **3**. Το δεύτερο 2 απορρίπτεται, άρα το σύνολο είναι $\{2, 4, 6\}$.

6.3.2 Ιούνιος 2026, Θέμα 2δ: στοίβα με push και pop σε βρόχο

Θέμα εξέτασης

d.

```
Stack<Integer> stack = new Stack<>();
for (int i = 3; i > 0; i--) {
    stack.push(i);
```

```
    stack.push(i-1);
    System.out.print(stack.pop() + " ");
}
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **2 1 0** (με κενά μετά από κάθε τιμή). Σε κάθε βήμα η στοίβα παίρνει δύο `push`, και το `pop` βγάζει το τελευταίο. Το υπόλοιπο μένει στη στοίβα για τα επόμενα βήματα.

7

Αριθμητικοί τύποι και πίνακες

Μέθοδος

(μέθοδος με δικά μας λόγια· βλ. eClass, διαλέξεις για τύπους δεδομένων και πίνακες)
Ακέραια διαίρεση. Η διαίρεση δύο `int` κόβει το δεκαδικό μέρος. Το `1 / a` με `int a` δίνει 0 για κάθε $|a| > 1$. Για δεκαδικό αποτέλεσμα χρειάζεται `1.0 / a`. Και η διαίρεση με μηδέν σε `int` ρίχνει εξαίρεση.
Προεπιλεγμένες τιμές. Οι αριθμητικοί πίνακες αρχικοποιούνται με 0, οι `boolean` με `false`, οι αναφορές με `null`. Αυτό ισχύει και για πεδία κλάσης.
Μετα- και προ-αύξηση. Το `j = i++` δίνει στο `j` την **παλιά** τιμή του `i` και μετά αυξάνει το `i`. Το `++i` αυξάνει πρώτα.
Πίνακες. Το `arr.length` είναι πεδίο, όχι μέθοδος. Η πρόσβαση εκτός ορίων δίνει `ArrayIndexOutOfBoundsException`. Η αντιστροφή ενός πίνακα με $n/2$ ανταλλαγές χρειάζεται `arr.length / 2` επαναλήψεις. Η `Arrays.toString` θέλει `import java.util.Arrays;`.

Παράδειγμα: ακέραια διαίρεση και επέκταση συνάρτησης (Ιούνιος 2025, Θέμα 3)

Θέμα εξέτασης

Θέμα 3ο (1,5 μονάδες): Τι ζητήματα έχει ο παρακάτω κώδικας; Υλοποιήστε μια επέκταση της παρακάτω μεθόδου ώστε να λειτουργεί σωστά. Υλοποιήστε σε Java ή C++.

```
double reciprocal(int a) {
    return 1 / a;
}
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Τα ζητήματα είναι δύο: η ακέραια διαίρεση (το αποτέλεσμα κόβεται) και η διαίρεση με μηδέν για `a == 0`. Μία λύση είναι:

```
double reciprocal(int a) {
    if (a == 0)
      throw new IllegalArgumentException("Division by zero: 0 has no reciprocal");
    return 1.0 / a;
}
```

Η απάντηση ελέγχθηκε μόνο με το χέρι, και η εκφώνηση δεν ορίζει τι γίνεται με το μηδέν.

Παράδειγμα: αντιστροφή πίνακα (Ιούνιος 2025, Θέμα 1β)

Θέμα εξέτασης

b.

```
int[] arr = {1, 3, 2, 4, 5};
for (int i = 0; i < arr.length / 2; i++) {
```

```
    int temp = arr[i];
    arr[i] = arr[arr.length - 1 - i];
    arr[arr.length - 1 - i] = temp;
}
System.out.println(Arrays.toString(arr));
```

Λύση
AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **[5, 4, 2, 3, 1]**. Οι δύο ανταλλαγές είναι $(1 \leftrightarrow 5)$ και $(3 \leftrightarrow 4)$. Ο κώδικας χρειάζεται `import java.util.Arrays;`.

Ακόμη λυμένα θέματα

7.3.1 Ιούνιος 2022, Ερώτηση 36 / 40: αριθμητική δεικτών (C++)

Θέμα εξέτασης

Ποιο είναι το αποτέλεσμα μετά από την εκτέλεση του παρακάτω κώδικα;

```
#include <iostream>

int
main (int argc, const char *argv[])
{
  int a[] = { 1, 2, 3, 4, 5, 6 };
  std::cout << (1 + 3)[a] - a[0] + (a + 1)[2];
}
```

- ☐ 6
- ☐ 7
- ☐ 8
- ☐ 9

Λύση
✓ **Κοινοτική λύση** (από το `2021-06.solutions.md`, με τον ίδιο κώδικα). Απάντηση: **8**. Η έκφραση $(1+3)[a]$ είναι $a[4] = 5$, το $a[0] = 1$ και το $(a+1)[2] = a[3] = 4$, άρα $5 - 1 + 4 = 8$. Το αρχείο λύσεων του 2022 δεν έχει λύση για αυτή την ερώτηση.

7.3.2 Σεπτέμβριος 2025, Θέμα 2γ: προεπιλεγμένες τιμές πίνακα

Θέμα εξέτασης

c.

```
boolean[] arr = new boolean[5];
System.out.println(arr[4]);
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **false**. Οι πίνακες `boolean` αρχικοποιούνται με `false`.

8

Πολυμορφισμός

Μέθοδος

(μέθοδος με δικά μας λόγια· βλ. eClass, διαλέξεις για πολυμορφισμό και δυναμική δέσμευση)
Κανόνας. Η **μέθοδος** που θα τρέξει καθορίζεται από τον **δυναμικό** τύπο του αντικειμένου, όταν η μέθοδος είναι virtual (C++) ή όταν είναι στιγμιότυπου (Java). Η **αναφορά** καθορίζει ποιες μεθόδους **επιτρέπεται** να καλέσεις (στατικός τύπος). Άρα: δείκτης (base) → αντικείμενο (derived) καλεί τη μέθοδο της derived.
Virtual στη C++. Χωρίς `virtual`, η επιλογή γίνεται με τον **στατικό** τύπο (δεσμεύεται νωρίς). Με `virtual`, γίνεται με τον δυναμικό (δεσμεύεται αργά). Κάθε κλάση με virtual μέθοδο έχει vTable, και κάθε αντικείμενο έχει vptr που τον δείχνει.
Καταστροφέας. Σε βάση με πολυμορφισμό, ο destructor πρέπει να είναι `virtual`, αλλιώς το `delete` μέσω δείκτη βάσης δεν καλεί τον καταστροφέα της παραγόμενης.
Υπερφόρτωση και υπερκάλυψη. Η υπερκάλυψη (overriding) είναι ίδια υπογραφή σε υποκλάση, και ο πολυμορφισμός εφαρμόζεται πάνω της. Η υπερφόρτωση (overloading) είναι ίδιο όνομα με διαφορετικές παραμέτρους στην ίδια κλάση.

Παράδειγμα: virtual show με δείκτη βάσης (Ιούνιος 2023, C++ 4/10)

Θέμα εξέτασης

Τι θα τυπώσει ο παρακάτω κώδικας;

```
#include<iostream>
using namespace std;

class Base {
public:
    virtual void show() { cout<<" In Base \n"; }
};

class Derived: public Base {
public:
    void show() { cout<<"In Derived \n"; }
};

int main(void) {
    Base *bp = new Derived;
    bp->show();
    return 0;
}
```

- ☐ In Base
- ☐ In Derived
- ☐ In Derived In Base
- ☐ In Base In Derived

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **"In Derived"** (2η επιλογή). Η `show` είναι virtual και ο δείκτης δείχνει σε `Derived`, άρα τρέχει η `Derived::show`.

Παράδειγμα: virtual destructor και κλήσεις (Ιούνιος 2023, C++ 9/10)

Θέμα εξέτασης

Τι θα τυπώσει ο παρακάτω κώδικας;

```
#include <iostream>

class Base {
    virtual void method() {std::cout << "from Base" << std::endl;}
public:
    virtual ~Base() {method();}
    void baseMethod() {method();}
};

class A : public Base {
    void method() {std::cout << "from A" << std::endl;}
public:
    ~A() {method();}
};

int main(void) {
    Base* base = new A;
    base->baseMethod();
    delete base;
    return 0;
}
```

- ☐ from A from Base from A
- ☐ from Base from Base from A
- ☐ from Base from A from Base
- ☐ from A from A from Base

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **from A, from A, from Base** (4η επιλογή). Η `baseMethod` καλεί την `method` με δυναμική δέσμευση, άρα "from A". Στο `delete`, πρώτα τρέχει ο `~A` ("from A"), και μετά ο `~Base`, όπου ο δυναμικός τύπος είναι πλέον `Base` ("from Base").

Παράδειγμα: υπερκάλυψη με δείκτη βάσης (Ιούνιος 2021, Ερώτηση 17 / 40)

Θέμα εξέτασης

Μετά την εκτέλεση του παρακάτω κώδικα στο παράθυρο εξόδου εμφανίζεται :

```
public class Parent {
    public void print()
    {
        System.out.println("Parent");
    }
}

public class Child extends Parent {
    public void print()
    {
        System.out.println("Child");
    }
}

public class Main {
    public static void main(String[] args)
    {
        Parent x = new Child();
        x.print();
    }
}
```

- ☐ Parent Child
- ☐ Parent
- ☐ Child
- ☐ Child Parent

Λύση

✓ **Κοινοτική λύση.** Απάντηση: **Child** (`2021-06.solutions.md`). Το `print` της υποκλάσης υπερκαλύπτει εκείνο της βάσης, και η αναφορά `Parent` δεν αλλάζει τη μέθοδο που τρέχει.

Ακόμη λυμένα θέματα

8.4.1 Ιούνιος 2025, Θέμα 5 (ii): πολυμορφισμός

Θέμα εξέτασης

ii. Μπορούμε να έχουμε πολυμορφισμό όταν χρησιμοποιούμε: - [ ] a. Δείκτη κλάσης βάσης σε αντικείμενο της κλάσης βάσης - [ ] b. Δείκτη παραγόμενης κλάσης σε αντικείμενο της παραγόμενης κλάσης - [ ] c. Δείκτη κλάσης βάσης σε αντικείμενο της παραγόμενης κλάσης - [ ] d. Δείκτη παραγόμενης κλάσης σε αντικείμενο της κλάσης βάσης

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **c**, δηλαδή δείκτης βάσης σε αντικείμενο παραγόμενης. Ο πολυμορφισμός θέλει τον στατικό τύπο της βάσης και τον δυναμικό της παραγόμενης.

8.4.2 Ιούνιος 2023, C++ 1/10: `getType` με `override`

Θέμα εξέτασης

Τι τυπώνει ο παρακάτω κώδικας;

```
#include <iostream>
using namespace std;
class Shape {
    public:
        int width = 1;
        int height = 2;
        virtual void getType() {
            cout << "Shape"<<endl;
        }
};
class Rectangle: public Shape {
    public:
        int side = 3;
        void getType () override {
            cout << "Rectangle"<<endl;
        }
};
int main() {
    Shape* s ;
    Rectangle r ;
    s = &r;
    s->getType();
    r.getType();
    return 0;
}
```

- ☐ Shape Shape
- ☐ Rectangle Rectangle
- ☐ Rectangle Shape
- ☐ Shape Rectangle

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **Rectangle, Rectangle** (2η επιλογή). Η `getType` είναι virtual, άρα και η κλήση μέσω `Shape*` καλεί τη `Rectangle::getType`, και η κλήση στο `r` καλεί την ίδια.

9

Θεωρία και ερωτήσεις πολλαπλής επιλογής

Μέθοδος

(μέθοδος με δικά μας λόγια· βλ. eClass, διαλέξεις εισαγωγής και C++)
Ερωτήσεις θεωρίας. Οι ερωτήσεις πολλαπλής επιλογής θέλουν ακρίβεια στη διατύπωση. Διάβασε κάθε επιλογή και ρώτα αν η λέξη-κλειδί (π.χ. “μόνο”, “πάντα”, “ποτέ”) κάνει την πρόταση λάθος.
C++. Η δήλωση κλάσης τελειώνει με `;`. Η λειτουργία του μέλους εκτός εμβέλειας δηλώνεται με τον τελεστή επίλυσης εμβέλειας `::`. Η διαδρομή μεταγλώττισης είναι προεπεξεργαστής → μεταγλωττιστής → συνδέτης (linker), και ο συνδέτης ενώνει τις βιβλιοθήκες.
Java. Ο κύκλος ζωής ενός applet είναι `init()`, `start()`, `paint()`. Ένα νήμα είναι αντικείμενο της `Thread` ή υποκλάσης της, και το `run()` είναι το περιεχόμενο της εργασίας του.

Παράδειγμα: operator scope σε C++ (Ιούνιος 2023, Θεωρία C++ 6/10)

Θέμα εξέτασης

Ποιο σύμβολο χρησιμοποιείται για τον ορισμό μέλους κλάσης εκτός εμβέλειάς της;

- ☐ :
- ☐ ::
- ☐ #
- ☐ ~

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **::** (2η επιλογή), ο τελεστής επίλυσης εμβέλειας.

Παράδειγμα: ερώτηση για νήμα (Ενδεικτική εξέταση 2023, Θεωρία 2/4)

Θέμα εξέτασης

Ένα νήμα είναι

- ☐ στιγμιότυπο της Runnable
- ☐ στιγμιότυπο της Thread
- ☐ στιγμιότυπο υποκλάσης της Thread
- ☐ στιγμιότυπο υποκλάσης της Runnable

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **η 2η επιλογή** (χαμηλή εμπιστοσύνη): η `Runnable` είναι η εργασία, ενώ η `Thread` είναι το αντικείμενο που την τρέχει. Η 3η επιλογή είναι επίσης υπερασπίσιμη.

Ακόμη λυμένα θέματα

9.3.1 Ιούνιος 2021, Ερώτηση 39 / 40: αναδρομική κλήση της `main`

Θέμα εξέτασης

Ποιο είναι το πρόβλημα στον παρακάτω κώδικα;

```
#include <iostream>
using namespace std;

int main()
{
int c = 0;

if ( ( c = cin.get() ) != EOF )
{
main();
cout << c;
}
}
```

- ☐ Λείπει ένα } στο τέλος
- ☐ Δεν έχει οριστεί το EOF
- ☐ Θα πρέπει αρχικά c >0
- ☐ Δεν επιτρέπεται αναδρομική κλήση της main

Λύση

✓ **Κοινοτική λύση.** Απάντηση: **Δεν επιτρέπεται αναδρομική κλήση της main** (`2021-06.solutions.md`). Στη C++ η `main` δεν μπορεί να καλείται από τον κώδικα του προγράμματος.

9.3.2 Ιούνιος 2023, Θεωρία C++ 3/10: δήλωση κλάσης

Θέμα εξέτασης

Ποιο από τα παρακάτω είναι σωστή δήλωση κλάσης;

- ☐ class A { int x; };
- ☐ class B { }
- ☐ public class A { }
- ☐ object A { int x; };

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **`class A { int x; };`** (1η επιλογή). Η C++ θέλει `;` μετά το `}`, και δεν έχει `public class` ή `object`.

10

Μνήμη, δημιουργία και καταστροφή αντικειμένων

Μέθοδος

(μέθοδος με δικά μας λόγια· βλ. eClass, διαλέξεις για συλλογή σκουπιδιών και δείκτες)
Στοίβα και σωρός. Ένα αντικείμενο που δημιουργείται με new μπαίνει στον **σωρό** (heap) και **δεν** καταστρέφεται όταν βγει εκτός εμβέλειας. Ένα **αυτόματο** αντικείμενο (χωρίς new) μπαίνει στη **στοίβα** και καταστρέφεται στο τέλος του μπλοκ του.
delete στη C++. Το `delete ptr` καλεί πρώτα τον καταστροφέα (destructor) και μετά ελευθερώνει τη μνήμη. Χωρίς `delete` ο καταστροφέας **δεν** τρέχει ποτέ, και έχεις διαρροή μνήμης.
Java. Δεν υπάρχει `delete`. Η συλλογή σκουπιδιών (garbage collection) ελευθερώνει αντικείμενα που δεν έχουν πια αναφορές. Αυτό δεν είναι το ίδιο με τον καταστροφέα της C++.
Στατικά αντικείμενα. Καταστρέφονται στο τέλος του προγράμματος, όχι στην έξοδο από εμβέλεια.

Παράδειγμα: new χωρίς delete (Ιούνιος 2022, Ερώτηση 8)

Θέμα εξέτασης

8) Τι τυπώνει ο παρακάτω κώδικας;

```
#include <iostream>

#include <memory>

class Resource
{
public:
Resource() { std::cout << "Constructor\n"; }
~Resource() { std::cout << "Destructor\n"; }
};

int main()
{
Resource* res = new Resource();

return 0;
}
```

- ☐ a) Constructor Destructor
- ☐ b) Constructor
- ☐ c) Destructor
- ☐ d) Τίποτα από τα παραπάνω

Λύση

✓ **Κοινοτική λύση.** Απάντηση: **b) Constructor** (`2022-06.solutions.md`). Ο κατασκευαστής τρέχει, αλλά δεν υπάρχει `delete`, άρα ο καταστροφέας δεν καλείται ποτέ.

Παράδειγμα: καταστροφέας στο τέλος της `main` (Ιούνιος 2026, Θέμα 5δ)

Θέμα εξέτασης

d. Τι θα τυπώσει ο παρακάτω κώδικας;

```
class Test {
public:
    Test() { cout << "C"; }
    ~Test() { cout << "D"; }
};

int main() {
    Test t;
    cout << "M";
}
```

- ☐ a. CMD
- ☐ b. CDM
- ☐ c. CM
- ☐ d. Τίποτα από τα παραπάνω

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **CMD** (a). Το `t` είναι αυτόματο, και ο καταστροφέας του τρέχει στο τέλος της `main`, μετά το `M`.

Παράδειγμα: καταστροφέας της `Test` στην εξέταση (Ενδεικτικό 2023, Θεωρία 4/4)

Θέμα εξέτασης

Τι ισχύει από τα παρακάτω;

- ☐ Ο destructor καλείται αυτόματα όταν ένα αντικείμενο βγει εκτός εμβέλειας
- ☐ Ο destructor καλείται αυτόματα όταν ένα static αντικείμενο βγει εκτός εμβέλειας
- ☐ Ο constructor καλείται αυτόματα όταν ένα αντικείμενο δημιουργείται
- ☐ Ο constructor καλείται όταν ένα αντικείμενο τον καλεί ρητά

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **η 3η επιλογή** με χαμηλή εμπιστοσύνη. Η 1η ισχύει για αυτόματα αντικείμενα αλλά όχι για δυναμικά, και η 2η είναι λάθος, γιατί τα static καταστρέφονται στο τέλος του προγράμματος.

Ακόμη λυμένα θέματα

10.4.1 Σεπτέμβριος 2025, Θέμα 5 (iv): `delete`

Θέμα εξέτασης

iv. Τι από τα παρακάτω ισχύει για ένα αντικείμενο που δημιουργείται δυναμικά; - [ ] Α. Ο τελεστής delete καλεί τον destructor και απελευθερώνει τη μνήμη - [ ] Β. Ο destructor καλεί τον τελεστή delete και απελευθερώνει τη μνήμη - [ ] Γ. Ο destructor απελευθερώνει τη μνήμη - [ ] Δ. Ο destructor καλεί τον τελεστή delete και η μνήμη παραμένει δεσμευμένη

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **Α**, από το αρχείο λύσεων του Σεπτεμβρίου 2025. Ο `delete` καλεί τον καταστροφέα και μετά ελευθερώνει τη μνήμη. Το αρχείο δεν έχει επαληθευτεί.

11

Αναφορές και συναρτήσεις (C++)

Μέθοδος

(μέθοδος με δικά μας λόγια· βλ. eClass, διαλέξεις για C++ και συναρτήσεις)
Αναφορά ως όρισμα. Η `void f(int &x)` δουλεύει με το **ίδιο** αντικείμενο: κάθε αλλαγή στο `x` αλλάζει τη μεταβλητή του καλούντα. Με τιμή (`int x`), αλλάζει μόνο αντίγραφο.
Προεπιλεγμένα ορίσματα. Τα προεπιλεγμένα ορίσματα μπαίνουν **στο τέλος** της λίστας. Η κλήση γεμίζει από αριστερά, άρα `sum(10)` σημαίνει `y = 2` και `z = 3`.
Δήλωση με δείκτες. Ένας πίνακας ως όρισμα γίνεται δείκτης. Η δήλωση `int *f(float *, char **)` είναι συνάρτηση που επιστρέφει δείκτη σε `int`. Διάβασε τον τύπο από δεξιά προς τα αριστερά, από το όνομα της συνάρτησης.
Υπερφόρτωση. Διαφορετικές παράμετροι, ίδιο όνομα. Δεν αρκεί να αλλάξει μόνο ο τύπος επιστροφής.

Παράδειγμα: αναφορά ως όρισμα (Ιούνιος 2026, Θέμα 5α)

Θέμα εξέτασης

a. Τι εκτυπώνει ο παρακάτω κώδικας; Αν ο κώδικας δεν μεταγλωττίζεται, εξηγήστε γιατί.

```
void f(int &x) {
    x = x + 2;
    cout << x;
}

int a = 5;
f(a);
cout << a;
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **77**. Η `x` είναι αναφορά στο `a`, άρα το `x = 7` γράφει στο `a` και τυπώνεται 7 δύο φορές.

Παράδειγμα: προεπιλεγμένα ορίσματα (Ιούνιος 2026, Θέμα 5β)

Θέμα εξέτασης

b. Τι εκτυπώνει ο παρακάτω κώδικας; Αν ο κώδικας δεν μεταγλωττίζεται, εξηγήστε γιατί.

```
int sum(int x, int y = 2, int z = 3) {
    return x + y + z;
}

int main() {
    int s = sum(10) + sum(10, 20) + sum(10, 20, 30);
    cout << s;
```

```
    return 0;
}
```

Λύση
AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **108**. Το `sum(10) = 15`, το `sum(10, 20) = 33` και το `sum(10, 20, 30) = 60`, και το άθροισμα είναι 108.

Παράδειγμα: δήλωση συνάρτησης χωρίς τιμή (Ιούνιος 2023, C++ 2/10)

Θέμα εξέτασης

Ποια είναι η δήλωση της συνάρτησης zero που δέχεται ένα array ακεραίων και δεν επιστρέφει κάποια τιμή;

- ☐ int zero (int []);
- ☐ void zero (int);
- ☐ void zero (int *);
- ☐ int zero (int *);

Λύση
AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **`void zero (int *);`** (3η επιλογή). Ο πίνακας ακεραίων περνά ως `int *`, και η απουσία τιμής επιστροφής είναι `void`.

Ακόμη λυμένα θέματα

11.4.1 Ιούνιος 2021, Ερώτηση 31 / 40: δείκτες σε δείκτες

Θέμα εξέτασης

Ποια είναι σωστή δήλωση για μια συνάρτηση που δέχεται σαν όρισμα ένα δείκτη float, ένα δείκτη σε δείκτη char και επιστρέφει ένα δείκτη σε δείκτη int;

- ☐ int **fun(float**, char**)
- ☐ int *fun(float*, char*)
- ☐ int **fun(float*, char**)
- ☐ int ***fun(*float,** char)

Λύση
Χωρίς κλειδί. Η μέθοδος: ο τύπος επιστροφής διαβάζεται πρώτος, και κάθε * στην παράμετρο αυξάνει το επίπεδο του δείκτη. Διάβασε κάθε επιλογή με αυτόν τον κανόνα. Η απάντηση δεν δόθηκε εδώ.

12

Τελεστές και friend συναρτήσεις

Μέθοδος

(μέθοδος με δικά μας λόγια· βλ. eClass, διαλέξεις για υπερφόρτωση τελεστών)
Friend. Μια `friend` συνάρτηση έχει πρόσβαση σε **όλα** τα μέλη της κλάσης, ιδιωτικά και προστατευόμενα. Δεν είναι μέλος της κλάσης και δεν έχει `this`.
Τελεστής ως μέλος. Σε `a + b` με τελεστή μέλους, το αριστερό μέλος `a` είναι το αντικείμενο για το οποίο καλείται η μέθοδος (`*this`), και το δεξί μέλος είναι το όρισμα. Το δεξί μέλος δεν χρειάζεται να είναι friend.
Τι θυμάσαι. Η πρόσβαση στα private μέλη γίνεται από μέσα στην κλάση, από friend και από τις μεθόδους της. Όχι από ξένη κλάση, όχι από `main`.

Παράδειγμα: friend και πρόσβαση (Ιούνιος 2025, Θέμα 5 (i))

Θέμα εξέτασης

i. Μια friend συνάρτηση: - [ ] a. Μπορεί να έχει πρόσβαση μόνο σε public members μιας κλάσης - [ ] b. Μπορεί να έχει πρόσβαση μόνο σε private members μιας κλάσης - [ ] c. Μπορεί να έχει πρόσβαση μόνο σε protected members μιας κλάσης - [ ] d. Μπορεί να έχει πρόσβαση σε όλα τα members μιας κλάσης

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **d**. Η friend συνάρτηση έχει πρόσβαση σε όλα τα μέλη της κλάσης που την δηλώνει.

Παράδειγμα: τελεστής ως μέλος (Ιούνιος 2025, Θέμα 5 (v))

Θέμα εξέτασης

v. Όταν υπερφορτώνουμε έναν δυαδικό τελεστή ως μέλος κλάσης: - [ ] Α. Το αριστερό μέλος του τελεστή είναι το αντικείμενο για το οποίο καλείται - [ ] Β. Το δεξιά μέλος του τελεστή είναι το αντικείμενο για το οποίο καλείται - [ ] Γ. Το αριστερό μέλος του τελεστή είναι το όρισμα της συνάρτησης - [ ] Δ. Θα πρέπει να δηλωθεί ως friend

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **Α**. Στον τελεστή μέλους, το αριστερό μέλος είναι το αντικείμενο που καλεί τη μέθοδο.

Ακόμη λυμένα θέματα

12.3.1 Σεπτέμβριος 2025, Θέμα 5 (iii): τελεστής ως μέλος

Θέμα εξέτασης

iii. Όταν υπερφορτώνουμε έναν δυαδικό τελεστή ως μέλος κλάσης: - [ ] Α. Το αριστερό μέλος του τελεστή είναι το αντικείμενο για το οποίο καλείται - [ ] Β. Το δεξιά μέλος του τελεστή είναι το αντικείμενο για το οποίο καλείται - [ ] Γ. Το αριστερό μέλος του τελεστή είναι το όρισμα της συνάρτησης - [ ] Δ. Θα πρέπει να δηλωθεί ως friend

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **Α**, από το αρχείο λύσεων του Σεπτεμβρίου 2025. Η ερώτηση είναι ίδια με το Θέμα 5 (v) του Ιουνίου 2025.

12.3.2 Ιούνιος 2021, Ερώτηση 28 / 40: friend συνάρτηση

Θέμα εξέτασης

Μια friend συνάρτηση: - [ ] Μπορεί να έχει πρόσβαση μόνο σε public members μιας κλάσης - [ ] Μπορεί να έχει πρόσβαση μόνο σε private members μιας κλάσης - [ ] Μπορεί να έχει πρόσβαση μόνο σε protected members μιας κλάσης - [ ] Μπορεί να έχει πρόσβαση σε όλα τα members μιας κλάσης

Λύση

Χωρίς κλειδί. Η μέθοδος: διάβασε τι ορίζει η friend δήλωση, και μετά ξαναδιάβασε τις τέσσερις επιλογές. Η απάντηση δεν δόθηκε εδώ.

13

Εξαιρέσεις

Μέθοδος

(μέθοδος με δικά μας λόγια· βλ. eClass, διαλέξεις για διαχείριση εξαιρέσεων)
try / catch / finally. Το `try` εκτελείται μέχρι την πρώτη εξαίρεση. Τότε η ροή πηδά στο πρώτο `catch` που ταιριάζει, και το `finally` τρέχει **πάντα**. Τα υπόλοιπα του `try` μετά την εξαίρεση δεν εκτελούνται.
Σειρά των catch. Τα `catch` με την **υποκλάση** μπαίνουν πρώτα. Η `ArithmeticException` (της `2/0`) είναι υποκλάση της `RuntimeException`.
Πίνακες και null. Η πρόσβαση εκτός ορίων δίνει `ArrayIndexOutOfBoundsException`. Η μέθοδος πάνω σε `null` (π.χ. `names[1].length()`) δίνει `NullPointerException`.
Μετατροπή αλφαριθμητικού. Η `Integer.parseInt("5.6")` δίνει `NumberFormatException`, γιατί το κείμενο δεν είναι ακέραιος.

Παράδειγμα: try, catch και finally (Ιούνιος 2021, Java)

Θέμα εξέτασης

```java
public class Test2 {
    public static void main(String[] args) {
        try {
            System.out.println("Java");
            int i = 0;
            int y = 2/i;
            System.out.println("Java");
        }
        catch (RuntimeException ex) {
            System.out.println("Java");
        }
        finally {
            System.out.println("Finally block");
        }
    }
}
```

- ☐ JavaJavaJavaFinally block
- ☐ JavaJavaFinally block
- ☐ JavaJavaJava
- ☐ JavaJava

Λύση

✓ **Κοινοτική λύση.** Απάντηση: **Java, Java, Finally block** (δηλαδή η 2η επιλογή, σε μορφή γραμμών: `Java`, `Java`, `Finally block`) (`2021-06.solutions.md`). Το πρώτο `Java` τυπώνεται πριν από τη διαίρεση, το δεύτερο στο `catch`, και το `finally` τυπώνεται πάντα.

Παράδειγμα: εξαιρέσεις στο `printLength` (Ιούνιος 2026, Θέμα 4)

Θέμα εξέτασης

Θέμα 4ο (2 μονάδες): Συμπληρώστε τον κώδικα της συνάρτησης printLength, η οποία εκτυπώνει το μήκος της συμβολοσειράς (String) που βρίσκεται στη θέση pos του πίνακα names. Η μέθοδος πρέπει να χειρίζεται τις πιθανές εξαιρέσεις ArrayIndexOutOfBoundsException και NullPointerException και να εκτυπώνει κατάλληλο μήνυμα.

```
public class Main {
    public static void printLength(String[] names, int pos) {
        // Συμπληρώστε τον κώδικα
    }

    public static void main(String[] args) {
        String[] names = {"Alice", null, "Bob"};
        printLength(names, 0);
        printLength(names, 1);
        printLength(names, 5);
    }
}
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Η λύση είναι ένα `try` με δύο `catch`, με τη σειρά που δείχνει η εκφώνηση:

```
public static void printLength(String[] names, int pos) {
    try {
        System.out.println("Length: " + names[pos].length());
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Invalid position: " + pos);
    } catch (NullPointerException e) {
        System.out.println("Null element at position " + pos);
    }
}
```

Για τις τρεις κλήσεις της `main`, το `pos=0` τυπώνει μήκος 5, το `pos=1` δίνει μήνυμα NPE, και το `pos=5` μήνυμα εκτός ορίων. Η λύση δεν ελέγχθηκε με μεταγλώττιση.

Ακόμη λυμένα θέματα

13.3.1 Σεπτέμβριος 2025, Θέμα 4: ορισμός `setValue` με εξαίρεση

Θέμα εξέτασης

Χειριστείτε την εξαίρεση ArrayIndexOutOfBoundsException που προκύπτει αν η τιμή του index είναι εκτός των ορίων του πίνακα.

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Η λύση γράφει `try { arr[index] = value; ... } catch (ArrayIndexOutOfBoundsException e)` με μήνυμα «Invalid index». Το μήνυμα και ο έλεγχος αρνητικού δείκτη δεν ζητούνται από την εκφώνηση, όπως σημειώνει το αρχείο λύσεων.

14 Πρότυπα (templates) και γενικεύσεις

Μέθοδος

(μέθοδος με δικά μας λόγια· βλ. eClass, διαλέξεις για C++ και templates)

Πρότυπο. Το `template <class T>` ορίζει μια κλάση ή συνάρτηση για τυχαίο τύπο `T`. Ο τύπος δίνεται στην χρήση, σε αγκύλες: `mypair<int>`.

Πλήθος παραμέτρων. Πρόσεξε πόσες παραμέτρους τύπου ορίζει το πρότυπο. Αν έχει μία `T`, η χρήση `mypair<int, int>` είναι λάθος, και η σωστή γραφή είναι `mypair<int>`.

Γενικεύσεις. Τα πρότυπα (templates) είναι ο μηχανισμός της C++ για γενικές δηλώσεις. Η κληρονομικότητα και οι εικονικές συναρτήσεις είναι άλλοι μηχανισμοί.

Παράδειγμα: ένα πρότυπο με μία παράμετρο (Ιούνιος 2021, Ερώτηση 40 / 40)

Θέμα εξέτασης

Έστω η παρακάτω πρότυπη κλάση. Ποιο από τα παρακάτω είναι έγκυρη δημιουργία στιγμιοτύπου της κλάσης;

```
1 template <class T>
2 class mypair {
3     T values [2];
4   public:
5     mypair (T first, T second)
6     {
7       values[0]=first; values[1]=second;
8     }
9 };
10
```

- ☐ `mypair<int, int> myobject (1, 2);`
- ☐ `mypair<> myobject (1, 2);`
- ☐ `mypair<int> myobject (1, 2);`
- ☐ `mypair<int, int> myobject (1);`

Λύση

✓ **Κοινοτική λύση.** Απάντηση: **`mypair<int> myobject (1, 2);`** (`2021-06.solutions.md`). Το πρότυπο έχει μία παράμετρο τύπου, και ο κατασκευαστής θέλει δύο ορίσματα.

Παράδειγμα: γενικεύσεις (Σεπτέμβριος 2025, Θέμα 5 (i))

Θέμα εξέτασης

i. _____ χρησιμοποιείται/ούνται για προγραμματισμό με γενικεύσεις (generics) - [ ] Α. Η κληρονομικότητα - [ ] Β. Οι εικονικές συναρτήσεις - [ ] Γ. Τα πρότυπα - [ ] Δ. Κανένα από τα παραπάνω

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Απάντηση: **Γ. Τα πρότυπα** (από το αρχείο λύσεων του Σεπτεμβρίου 2025).

Ακόμη λυμένα θέματα

Δεν υπάρχουν άλλα λυμένα θέματα αυτής της ενότητας στις εξετάσεις. Δείτε την άσκηση εξάσκησης παρακάτω.

15

Ασκήσεις εξάσκησης

Λύσε τα παρακάτω σαν να είσαι στην εξέταση: χωρίς σημειώσεις και με χρονόμετρο. Η αρίθμηση είναι όπως τυπώνεται στη μεταγραφή (π.χ. «Ερώτηση 12 / 40») ή, όπου λείπει, η σειρά εμφάνισης.

Σήμανση: ✓ κοινοτική λύση (αναφέρεται το αρχείο λύσεων)· ⚙ λύση AI, ai-unverified, που δεν έχει ελεγχθεί από άνθρωπο· ○ χωρίς λύση.

Κλάσεις, αντικείμενα και κατασκευαστές (κεφ. 1)

- ⚙ 2020 (μήνας άγνωστος, Μέρος 1ο) — εφαρμογή Click Away, σχεδίαση κλάσεων (ai-unverified· λύση μόνο για Α.1 και Α.2, χωρίς μεταγλώττιση)
- ⚙ Ιούνιος 2026, Θέμα 3 — κλάση Course με κατασκευαστές `this(...)`
- ⚙ Ιούνιος 2026, Θέμα 2β — στατικό `count` σε κατασκευαστή
- ⚙ Ιούνιος 2023, Java 3/10 — `XWindow(4)` και στατικός μετρητής
- ⚙ Ιούνιος 2023, Java 4/10 — `XWindow(6)` και `--x`
- ⚙ Ιούνιος 2023, Java 8/10 — `biggerWindow` σε αντικείμενα της ίδιας κλάσης
- ✓ Ιούνιος 2021, Ερώτηση 12 / 40 — `this.code` και τοπική μεταβλητή
- ○ Ιούνιος 2021, Ερώτηση 18 / 40 — κλήση μεθόδου σε στιγμιότυπο (χωρίς λύση)
- ✓ Ιούνιος 2021, Ερώτηση 20 / 40 — στατικός μετρητής στην κλάση `Test4` (κοινοτική λύση: 3 3 3)
- ✓ Ιούνιος 2021 (χωρίς αριθμό) — «Μια κλάση ΔΕΝ μπορεί να έχει ως προσδιοριστή το» (κοινοτική λύση: final)
- ⚙ Σεπτέμβριος 2025, Θέμα 6 (i) — προτιμώμενος constructor για `time` (ai-unverified· λύση Α)
- ⚙ Σεπτέμβριος 2025, Θέμα 3 — κλάση `Book` με ονομασμένους κατασκευαστές (ai-unverified· η λύση υλοποιεί την `Student`, άρα δεν είναι αξιόπιστη)
- ⚙ Ενδεικτικό 2023, Θεωρία 3/4 — δείκτης `this`
- ⚙ Ιούνιος 2026, Θέμα 5γ — `const` μέλος και λίστα αρχικοποίησης
- ⚙ Σεπτέμβριος 2025, Θέμα 2β — `private` πεδίο σε άλλη κλάση

Αναφορές, ισότητα και αλφαριθμητικά (κεφ. 2)

- ✓ Ιούνιος 2021, Ερώτηση 5 / 40 — `w1 = w2` (κοινοτική λύση: η w1 αντιπροσωπεύει το αντικείμενο της w2)
- ⚙ Ιούνιος 2026, Θέμα 1γ — == σε literal, `new String` και `equals`
- ⚙ Ιούνιος 2026, Θέμα 2α — `Example` με == σε δύο αντικείμενα
- ✓ Ιούνιος 2021, Ερώτηση 11 / 40 — `compString` με `void` και `equals`
- ✓ Ιούνιος 2021 (χωρίς αριθμό) — `Theory a==b` και `c=b` (κοινοτική λύση: false και true)
- ⚙ Ιούνιος 2023, Java 9/10 — `compString` με τη διόρθωση του `if !result`
- ⚙ Ιούνιος 2023, Java 2/10 — `charCount` με `charAt`
- ⚙ Ιούνιος 2023, Java 1/10 — `changeString` με `replace` (η `String` είναι αμετάβλητη, άρα «Dagat»)
- ⚙ Ενδεικτικό 2023, Προγραμματισμός 2/4 — `changeString` με `snew`

Κληρονομικότητα (κεφ. 3)

- ⚙ Ιούνιος 2025, Θέμα 2δ — κλήση μεθόδου της υποκλάσης μέσω υπερκλάσης
- ⚙ Ιούνιος 2026, Θέμα 2γ — κλήση `g()` σε αναφορά `A`
- ⚙ Ιούνιος 2023, Java 5/10 — `EWindow(3.5)` και `size--` (η σωστή έξοδος δεν ταιριάζει με καμία επιλογή, βλ. σημείωση)
- ⚙ Ιούνιος 2023, C++ 8/10 — `Shape s = r` (slicing)
- ⚙ Ιούνιος 2023, C++ 10/10 — δείκτες `Rectangle` και `Shape` στο ίδιο αντικείμενο

- ✓ Ιούνιος 2021, Ερώτηση 22 / 40 — constructor της υπερκλάσης
- ✓ Ιούνιος 2021 (χωρίς αριθμό) — `Student` και `MasterStudent` (κοινοτική σήμανση «run-time error»· το αρχείο λύσεων σημειώνει ότι η σήμανση πρέπει να ελεγχθεί, γιατί η ανάθεση είναι έγκυρη)
- ✓ Ιούνιος 2021, Ερώτηση 19 / 40 — `XWindow(4)` και `x++` (κοινοτική λύση: 4)
- ☼ Ενδεικτικό 2023, Πρόγραμμα 1ο — `dmode` και κρυμμένο πεδίο

Συλλογές Java (κεφ. 4)

- ☼ Ιούνιος 2025, Θέμα 1α — `list.add(1, 20)`
- ☼ Ιούνιος 2025, Θέμα 1γ — `remove("Charlie")` σε `ArrayList<String>`
- ☼ Ιούνιος 2025, Θέμα 2α — `Stack` με `pop` σε άδεια στοίβα (EmptyStackException)
- ☼ Ιούνιος 2025, Θέμα 2β — αφαίρεση μέσα σε βρόχο
- ☼ Σεπτέμβριος 2025, Θέμα 1α — `LinkedList` ως ουρά
- ☼ Σεπτέμβριος 2025, Θέμα 1β — `HashSet` με διπλότυπα
- ☼ Σεπτέμβριος 2025, Θέμα 2α — στοίβα με `push` και `pop` σε βρόχο
- ☼ Ιούνιος 2026, Θέμα 1α — `ArrayList.remove("A")` με διπλότυπα
- ☼ Ιούνιος 2026, Θέμα 1β — `HashSet` με `size`
- ☼ Ιούνιος 2026, Θέμα 2δ — στοίβα με δύο `push` σε βρόχο

Αριθμητικοί τύποι και πίνακες (κεφ. 5)

- ☼ Ιούνιος 2025, Θέμα 1β — αντιστροφή πίνακα
- ☼ Ιούνιος 2025, Θέμα 2γ — προεπιλεγμένες τιμές πίνακα `int`
- ☼ Ιούνιος 2025, Θέμα 3 — ακέραια διαίρεση στη `reciprocal`
- ✓ Ιούνιος 2022, Ερώτηση 36 / 40 — αριθμητική δεικτών (κοινοτική λύση: 8)
- ☼ Ιούνιος 2023, C++ 7/10 — τετράγωνα από 1 έως 10 και άθροισμα
- ☼ Ιούνιος 2026, Θέμα 1δ — προεπιλογή πίνακα `int`
- ☼ Σεπτέμβριος 2025, Θέμα 1δ — `continue` και `break` σε βρόχο `for`
- ☼ Σεπτέμβριος 2025, Θέμα 2γ — προεπιλεγμένες τιμές `boolean`
- ○ Ιούνιος 2021, Ερώτηση 33 / 40 — `int j = i++` (χωρίς λύση)

Πολυμορφισμός (κεφ. 6)

- ✓ Ιούνιος 2021, Ερώτηση 17 / 40 — `Parent x = new Child()`
- ✓ Ιούνιος 2021, Ερώτηση 9 / 40 — υπερκάλυψη μεθόδου (κοινοτική λύση: υπερκάλυψης)
- ○ Ιούνιος 2021, Ερώτηση 24 / 40 — δείκτες κλάσης βάσης και παραγόμενης (χωρίς λύση)
- ○ Ιούνιος 2021 (χωρίς αριθμό) — «Η ύπαρξη πολλών δημιουργών σε μια κλάση αποτελεί φαινόμενο» (χωρίς λύση στο αρχείο)
- ☼ Ιούνιος 2023, C++ 4/10 — `virtual show` με δείκτη βάσης
- ☼ Ιούνιος 2023, C++ 1/10 — `getType` με `override`
- ☼ Ιούνιος 2023, C++ 9/10 — virtual destructor
- ☼ Ιούνιος 2025, Θέμα 5 (ii) — πολυμορφισμός με δείκτη βάσης
- ☼ Σεπτέμβριος 2025, Θέμα 5 (v) — vptr και vTable (ai-unverified· η σημείωση του αρχείου θεωρεί τη σήμανση προς επανέλεγχο)

Θεωρία και ερωτήσεις πολλαπλής επιλογής (κεφ. 7)

- ✓ Ιούνιος 2021, Ερώτηση 39 / 40 — αναδρομική κλήση της `main`
- ☼ Ιούνιος 2023, Θεωρία C++ 6/10 — σύμβολο εκτός εμβέλειας (`::`)
- ☼ Ιούνιος 2023, Θεωρία C++ 3/10 — δήλωση κλάσης

- ⚙ Ενδεικτικό 2023, Θεωρία 1/4 — κύκλος ζωής applet
- ⚙ Ενδεικτικό 2023, Θεωρία 2/4 — νήμα
- ✓ Ιούνιος 2021 (χωρίς αριθμό) — «Η λειτουργία ενός νήματος» (κοινοτική λύση: `run()`)
- ✓ Ιούνιος 2021 (χωρίς αριθμό) — «Η εντολή File f1 = new File» (κοινοτική λύση: αντικείμενο-αρχείο)
- ⚙ Σεπτέμβριος 2025, Θέμα 5 (ii) — συνδέτης (linker)

Μνήμη, δημιουργία και καταστροφή αντικειμένων (κεφ. 8)

- ✓ Ιούνιος 2022, Ερώτηση 8 — `new` χωρίς `delete`
- ⚙ Ιούνιος 2023, C++ 3/10 — `Resource` με `new` και χωρίς `delete` (η ίδια λογική με το 2022)
- ⚙ Ενδεικτικό 2023, Θεωρία 4/4 — constructor και destructor
- ⚙ Ιούνιος 2025, Θέμα 5 (iv) — `new` και στοίβα ή σωρός
- ⚙ Σεπτέμβριος 2025, Θέμα 5 (iv) — `delete` και destructor
- ⚙ Σεπτέμβριος 2025, Θέμα 6 (ii) — `Resource` με `new` (ai-unverified· λύση b)
- ⚙ Ιούνιος 2026, Θέμα 5δ — καταστροφέας στο τέλος της `main`

Αναφορές και συναρτήσεις C++ (κεφ. 9)

- ⚙ Ιούνιος 2026, Θέμα 5α — αναφορά ως όρισμα (77)
- ⚙ Ιούνιος 2026, Θέμα 5β — προεπιλεγμένα ορίσματα (108)
- ⚙ Ιούνιος 2026, Θέμα 5γ — σωστός constructor για `const` μέλος
- ⚙ Ιούνιος 2023, C++ 2/10 — δήλωση συνάρτησης `zero`
- ⚙ Ενδεικτικό 2023, Προγραμματισμός 4/4 — δήλωση συνάρτησης με δείκτες
- ○ Ιούνιος 2021, Ερώτηση 31 / 40 — δείκτες σε δείκτες (χωρίς λύση)
- ○ Ιούνιος 2021, Ερώτηση 32 / 40 — δήλωση `zero` με `void` (χωρίς λύση)

Τελεστές και friend συναρτήσεις (κεφ. 10)

- ⚙ Ιούνιος 2025, Θέμα 5 (i) — friend και πρόσβαση
- ⚙ Ιούνιος 2025, Θέμα 5 (v) — τελεστής ως μέλος
- ⚙ Σεπτέμβριος 2025, Θέμα 5 (iii) — τελεστής ως μέλος (ai-unverified)
- ○ Ιούνιος 2021, Ερώτηση 28 / 40 — friend συνάρτηση (χωρίς λύση)

Εξαιρέσεις (κεφ. 11)

- ✓ Ιούνιος 2021 (χωρίς αριθμό) — try, catch και finally (κοινοτική λύση: δύο γραμμές «Java» και «Finally block»)
- ⚙ Ιούνιος 2026, Θέμα 4 — `printLength` με `try` και `catch`
- ⚙ Σεπτέμβριος 2025, Θέμα 4 — `setValue` με `ArrayIndexOutOfBoundsException` (ai-unverified· η λύση προσθέτει έλεγχο αρνητικού δείκτη)

Πρότυπα (templates) και γενικεύσεις (κεφ. 12)

- ✓ Ιούνιος 2021, Ερώτηση 40 / 40 — `mypair<int>`
- ⚙ Σεπτέμβριος 2025, Θέμα 5 (i) — πρότυπα ως γενικεύσεις

Σημείωση

Προσομοίωση. Η πιο πλούσια εξέταση είναι **Ιούνιος 2026** (θέμα 1 έως 5, με κώδικα). Διάλεξε 3 θέματα, σε 90 λεπτά (η διάρκεια της εξέτασης δεν είναι γνωστή), και γράψε **όλα** τα βήματα ιχνηλάτησης, κυρίως στα προγράμματα με στατικές μεταβλητές και αναφορές. Μετά σύγκρινε μόνο με το κεφάλαιο.

Πηγές και ευχαριστίες

- **Θέματα:** οι διδάσκοντες του μαθήματος. Παρατίθενται αυτούσια για σκοπούς μελέτης· τα δικαιώματα ανήκουν στους δημιουργούς τους.
- **Μεταγραφές:** η βάση γνώσης CEID (8 εξετάσεις, 2020 έως 2026). Όλες είναι μερικές. Το Ιούνιος 2021 έχει 23 από τις 40 ερωτήσεις (16 με ορατό αριθμό), το Ιούνιος 2022 έχει δύο ερωτήσεις, το Ιούνιος 2023 είναι στιγμιότυπα της πλατφόρμας eClass με διπλότυπες σελίδες, και το Σεπτέμβριος 2025 και το Ιούνιος 2026 έχουν λείπουσες σελίδες.
- **Κοινοτικές λύσεις:** αρχεία λύσεων της βάσης για Ιούνιο 2021 (από λυμένη έκδοση άγνωστης προέλευσης) και Ιούνιο 2022 (από φωτογραφία με σημείωση φοιτητή). Η κατάσταση είναι `community` και όχι επίσημη.
- **Λύσεις AI χωρίς έλεγχο:** για τις εξετάσεις 2020, 2023 (Ιούνιος και ενδεικτικό), 2025 (Ιούνιος) και 2026 (Ιούνιος), καθώς και για τη μέθοδο του αρχείου Σεπτεμβρίου 2025 (ai-unverified). Όλες σημαίνονται με ⊙.
- **Ενδεικτικό τεστ 2023:** δεν είναι βαθμολογούμενη εξέταση, και στο τεστ προγραμματισμού λείπει μία ερώτηση.
- **Γνώμες φοιτητών:** το κανάλι του μαθήματος στο Discord του CEID, ανώνυμα και με ημερομηνία.
- **Περίγραμμα:** Περίγραμμα μαθήματος (ap-ceid_nny106_perigrammamathimatos_gr_1.pdf) και κατάλογος μαθημάτων 2025-26, με ECTS 9, τομέα ΛΥ και διδάσκοντες 2025-26. Το κείμενο του περιγράμματος αναφέρει την ύλη και τη βιβλιογραφία.

Κείμενο, επεξηγήσεις και έλεγχος λύσεων: Giorgos Kandilas, με τη βοήθεια AI.

Βρήκες λάθος ή έχεις επίσημες λύσεις ή θέματα που λείπουν (π.χ. Ιανουάριος 2026, Σεπτέμβριος 2026); Στείλ' τα· κάθε συνεισφορά αναφέρεται στην επόμενη έκδοση.

Πώς να συνεισφέρεις: διορθώσεις, θέματα που λείπουν και επίσημες λύσεις δέχονται στο `https://kdngiorgos.github.io/ceid-notes/` — κάθε συνεισφορά αναφέρεται στην επόμενη έκδοση.